

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет шаурмечного дела

Машинно-зависимые языки и основы компиляции

Конспект ответов на вопросы к экзамену

ПОЛНЫЙ СВОД ОТВЕТОВ НА ВОПРОСЫ

Выполнил студент:

Студент-красавчик

Группа: Группа ИУ6-7

Содержание

Раздел 1. Архитектура микропроцессоров и программирование на языке ассемблера 4

1. Процессор i8086. Структура, основные регистры и взаимодействие частей в процессе функционирования.	4
2. Процессор i8086. Адресация оперативной памяти i8086.	5
3. Процессор i8086. Адресация сегментов различных типов.	6
4. Процессор i8086. Структура машинной команды. Примеры.	6
5. Процессор IA-32. Программная модель.	6
6. Процессор IA-32. Режимы адресации оперативной памяти. Схема адресации в защищенном режиме.	6
7. Процессор IA-32. Структура машинной команды. Байты изменения длины операнда и адреса. Примеры.	6
8. Структура программы на ассемблере Nasm. Директивы резервирования памяти и объявления данных. Символическая адресация данных. Примеры.	6
9. Процедуры. Примеры.	6
10. Передача параметров в процедуру через глобальные переменные при совместной трансляции модулей. Пример.	6
11. Передача параметров в процедуру через глобальные переменные при отдельной трансляции модулей. Пример.	6
12. Передача параметров в процедуру через таблицу. Пример.	7
13. Передача параметров в процедуру через стек. Пример.	7
14. Преобразование данных при вводе для консольного режима. Пример.	7
15. Преобразование данных для вывода в консольном режиме. Пример.	7
16. Организация многомодульных программ на Nasm. Пример.	7
17. Проблемы связи разноязыковых модулей. Пример.	7
18. Связь Pascal – Nasm. Конвенция. Пример.	7
19. Операнды команд ассемблера. Размеры операндов. Символическая адресация данных. Примеры.	7
20. Два способа адресации элементов массива. Примеры.	7
21. Связь Pascal – Nasm. Создание локальных переменных. Пример.	7
22. Связь C++ – Nasm. Конвенция. Пример.	8
23. Связь Pascal – Nasm. Процедура без параметров. Пример.	8
24. Связь C++ – Nasm. Создание внешних переменных. Пример.	8
25. Связь C++ – Nasm. Создание локальных переменных. Пример.	8

Раздел 2. Основы построения компиляторов 8

26. Проблема построения компилирующих программ. Метод Рутисхаузера. Пример.	8
27. Типы компилирующих программ и их особенности.	9
28. Структура компилирующей программы. Основные фазы процесса. Пример.	10
29. Синтаксис и семантика языка программирования. Средства описания синтаксиса. Примеры.	10
30. Синтаксические диаграммы и особенности их построения. Примеры.	11

31. Формальный язык и формальная грамматика языка. Пример.	11
32. Классификация грамматик Хомского. Примеры грамматик 2-го и 3-го типов.	11
33. Грамматический разбор. Дерево грамматического разбора. Пример.	11
34. Левосторонний нисходящий грамматический разбор для грамматик 3-го типа. Пример. . .	11
35. Левосторонний восходящий грамматический разбор для грамматик 3-го типа. Пример. ...	11
36. Конечный автомат. Пример описания и алгоритм программной реализации.	11
37. Построение лексических анализаторов на конечных автоматах. Пример.	11
38. Построение синтаксических анализаторов на конечных автоматах. Пример.	11
39. Автомат с магазинной памятью.	11
40. Левосторонний нисходящий грамматический разбор для грамматик 2-го типа. Пример. . .	12
41. Левосторонний восходящий грамматический разбор для грамматик 2-го типа. Пример. ...	12
42. LL(k) грамматики. Метод рекурсивного спуска. Пример.	12
43. LR(k) грамматики. Операторное предшествование. Стековый метод. Пример.	12
44. LR(k) грамматики. Стековый метод. Пример.	12
45. LR(k) грамматики. Польская запись. Алгоритм Бауэра-Замельзона. Пример.	12
46. Способы распределения памяти под переменные. Достоинства и недостатки.	12
47. Машинно-зависимая оптимизация кодов. Примеры.	12
48. Машинно-независимая оптимизация кодов. Примеры.	12

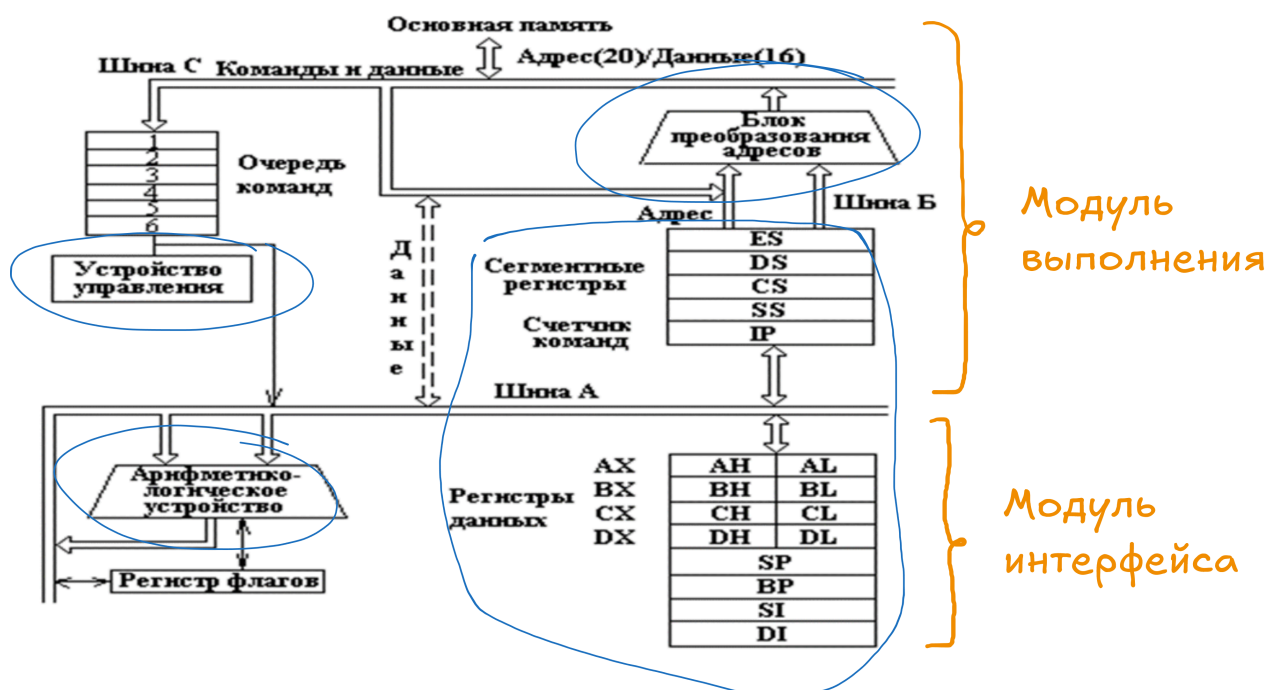
Раздел 1. Архитектура микропроцессоров и программирование на языке ассемблера

1. Процессор i8086. Структура, основные регистры и взаимодействие частей в процессе функционирования.

Процессор i8086 – родоначальник семейства процессоров Intel x86, который определил особенности, присущие процессорам этого ряда.

РОНы/Данные/Адрес: 16/16/20; Осн. хар-ки: 4-8 МГц, 1 Мб

Упрощенная структурная схема процессора i8086:



В общем случае процессор состоит из двух модулей – модуль выполнения и модуль интерфейса, а их взаимодействие происходит через шину данных.

В его состав входят: **устройство управления (УУ)**, **арифметико-логическое устройство (АЛУ)**, **блок преобразования (формирования) адресов** и **регистры**:

- **УУ** дешифрует коды команд и формирует необходимые управляющие сигналы.
- **АЛУ** осуществляет необходимые арифметические и логические преобразования данных.
- В блоке **преобразования** адресов формируются физические адреса данных, расположенных в основной памяти.
- **Регистры** – внутренняя память процессора – используются для хранения управляющей информации (в основном адресов и данных).

Всего в состав процессора i8086 входят четырнадцать 16-ти битовых регистров, каждый из которых может иметь специальное назначение, описанное далее:

1. **Четыре регистра общего назначения** (называемые также регистрами данных):
 - **AX** (Accumulator) – регистр-аккумулятор (для умножения, деления (записывается целая часть), для команд ввода/вывода);
 - **BX** (Base) – базовый регистр;
 - **CX** (Counter) – счетчик (для циклов);
 - **DX** (Data) – регистр-расширитель аккумулятора (для остатка от деления, адрес ввода/вывода);
2. **Три адресных регистра**, которые должны использоваться для хранения частей адреса данных или применяется соответствующая команда:
 - **SI** (Source Index) – регистр индекса источника;
 - **DI** (Destination Index) – регистр индекса приемника;
 - **BP** (Base Pointer) – регистр-указатель базы стека;
3. **Три управляющих регистра**:
 - **SP** (Stack Pointer) – регистр-указатель стека;
 - **IP** (Instruction Pointer) – регистр-счетчик команд;
 - **Flags** – регистр флагов;
4. **Четыре сегментных регистра**:
 - **CS** (Code Segment) – регистр сегмента кодов;
 - **DS** (Data Segment) – регистр сегмента данных;
 - **ES** (Extra Segment) – регистр дополнительного сегмента данных;
 - **SS** (Stack Segment) – регистр сегмента стека.

2. Процессор i8086. Адресация оперативной памяти i8086.

Процессор i8086 – родоначальник семейства процессоров Intel x86, который определил особенности, присущие процессорам этого ряда.

РОНы/Данные/Адрес: 16/16/20; Осн. хар-ки: 4-8 МГц, 1 Мб

Минимальной адресуемой единицей основной памяти является **байт**, состоящий из **8 битов**. Память представляет собой последовательность байтов. Номер байта является его физическим адресом в устройстве памяти.

Важно: Адресовать отдельно бит нельзя. Если необходимо получить доступ к определенному биту, то сначала ищется соответствующий байт, а затем уже в нем – нужный бит.

В i8086 для адресации оперативной памяти используются адресная шина размером **20 бит**, осуществляется работа с памятью до **1 МБ**. Каждый сегментный регистр используется для хранения памяти. Адрес, задаваемый в регистре или команде, представлял собой не физический адрес памяти, а смещение в некоем сегменте относительно начального (базового) адреса этого сегмента. Начальные адреса всех используемых в данный момент сегментов задаются процессору в специальных сегментных регистрах.

Процессор i8086 содержит 16-ти разрядную модель памяти.

Схема адресации «база + смещение»: $A_{\phi} = A_6 + A$

3. Процессор i8086. Адресация сегментов различных типов.

Ожидает заполнения.

4. Процессор i8086. Структура машинной команды. Примеры.

Ожидает заполнения.

5. Процессор IA-32. Программная модель.

Ожидает заполнения.

6. Процессор IA-32. Режимы адресации оперативной памяти. Схема адресации в защищенном режиме.

Ожидает заполнения.

7. Процессор IA-32. Структура машинной команды. Байты изменения длины операнда и адреса. Примеры.

Ожидает заполнения.

8. Структура программы на ассемблере Nasm. Директивы резервирования памяти и объявления данных. Символическая адресация данных. Примеры.

Ожидает заполнения.

9. Процедуры. Примеры.

Ожидает заполнения.

10. Передача параметров в процедуру через глобальные переменные при совместной трансляции модулей. Пример.

Ожидает заполнения.

11. Передача параметров в процедуру через глобальные переменные при раздельной трансляции модулей. Пример.

Ожидает заполнения.

12. Передача параметров в процедуру через таблицу. Пример.

Ожидает заполнения.

13. Передача параметров в процедуру через стек. Пример.

Ожидает заполнения.

14. Преобразование данных при вводе для консольного режима. Пример.

Ожидает заполнения.

15. Преобразование данных для вывода в консольном режиме. Пример.

Ожидает заполнения.

16. Организация многомодульных программ на Nasm. Пример.

Ожидает заполнения.

17. Проблемы связи разноязыковых модулей. Пример.

Ожидает заполнения.

18. Связь Pascal – Nasm. Конвенция. Пример.

Ожидает заполнения.

19. Операнды команд ассемблера. Размеры операндов. Символическая адресация данных. Примеры.

Ожидает заполнения.

20. Два способа адресации элементов массива. Примеры.

Ожидает заполнения.

21. Связь Pascal – Nasm. Создание локальных переменных. Пример.

Ожидает заполнения.

22. Связь C++ – Nasm. Конвенция. Пример.

Ожидает заполнения.

23. Связь Pascal – Nasm. Процедура без параметров. Пример.

Ожидает заполнения.

24. Связь C++ – Nasm. Создание внешних переменных. Пример.

Ожидает заполнения.

25. Связь C++ – Nasm. Создание локальных переменных. Пример.

Ожидает заполнения.

Раздел 2. Основы построения компиляторов

26. Проблема построения компилирующих программ. Метод Рутисхаузера. Пример.

Первые компилирующие программы обеспечивали перевод формульной записи выражений в машинный язык. В основе такого перевода лежит представление выражения в виде последовательности **троек**, где каждая тройка включает два адреса операндов и операцию:

Пример: $A + B$, где A и B - операнды, $+$ - операция.

Основной проблемой при этом является необходимость учета приоритетов операций. Например, для выражения $a + b * c$ должны быть построены тройки $T_1 = b * c$, $T_2 = a + T_1$. Для решения этой задачи первым был предложен **метод Рутисхаузера**.

Метод Рутисхаузера требует, чтобы выражение было записано в полной скобочной записи, когда порядок выполнения операций указывается не правилами приоритета, а скобками. Так, выражение $d = a + b * c$ должно быть записано в виде $d = a + (b * c)$, в противном случае сначала будет выполняться операция сложения.

Метод заключается в следующем:

Шаг 1. Каждому символу строки выражения S_i ставится в соответствие индекс N_i по следующему алгоритму:

$N[0] := 0$

$J := 1$

Цикл-пока $S[J] \neq \text{'_'}$

Если $S[J] = \text{'('}$ или $S[J] = \text{<операнд>}$


```

    то N[J]:=N[J-1] + 1
    иначе N[J]:=N[J-1] - 1
Все-если
J:=J+1
Все-цикл
N[J]:=0

```

Шаг 2. Определяется наибольшее значение индекса k в строке индексов. Это значение входит в подстроку вида $k(k-1)k$ или при наличии скобок – в подстроку вида $(k-1)k(k-1)k(k-1)$. После этого строится соответствующая подстроке тройка.

Шаг 3. Обработанные символы вместе со скобками удаляются, и на их место ставится значение $n = k - 1$.

Шаг 4. Шаги 2, 3 повторяются до завершения преобразования выражения в последовательность троек.

Пример.

$$1) \quad S: ((a + b) * c) + d / k \quad T1 = (a + b)$$

$$N: 0 \ 1 \ 2 \ 3 \ 4 \ 3 \ 4 \ 3 \ 2 \ 3 \ 2 \ 1 \ 2 \ 1 \ 0 \ 1 \ 0$$

$$2) \quad S: (T1 * c) + d / k \quad T2 = (T1 * c)$$

$$N: 0 \ 1 \ 2 \ 3 \ 2 \ 3 \ 2 \ 1 \ 2 \ 1 \ 0 \ 1 \ 0$$

$$3) \quad S: (T2 + d) / k \quad T3 = (T2 + d)$$

$$N: 0 \ 1 \ 2 \ 1 \ 2 \ 1 \ 0 \ 1 \ 0$$

$$4) \quad S: T3 / k \quad T4 = T3 / k$$

$$N: 0 \ 1 \ 0 \ 1 \ 0$$

27. Типы компилирующих программ и их особенности.

Транслятор — это программа, которая осуществляет перевод программы, написанной на одном языке, в эквивалентную ей программу, написанную на другом языке

Компилятор — это разновидность транслятора, особенность которого заключается в переводе исходного кода с языка высокого уровня в машинный язык, язык ассемблера или в байт-код

Ассемблер — это транслятор, который специализируется на переводе программ с языка Ассемблера в машинный язык

Интерпретатор — это программа, главная особенность которой состоит в том, что она принимает исходную программу и сразу выполняет ее, не создавая при этом эквивалентной программы на другом языке

Макрогенератор (для компиляторов он выступает в роли препроцессора) — это программа, которая обрабатывает исходный код как простой текст и осуществляет в нем замену указанных символов на подстроки

Его ключевая особенность заключается в том, что он обрабатывает исходную программу до начала процесса трансляции

28. Структура компилирующей программы. Основные фазы процесса. Пример.

Структура процесса компиляции состоит из последовательных этапов обработки исходного текста программы. В предоставленных источниках выделяются следующие основные фазы:

1. **Лексический анализ:** процесс выделения отдельных слов (лексем) в предложениях языка и преобразования этих предложений в строку однородных символов, которые называются токенами.
2. **Синтаксический анализ:** процесс распознавания допустимых синтаксических конструкций языка в полученной строке токенов.
3. **Семантический анализ:** процесс распознавания и проверки логического смысла выделенных конструкций.
4. **Распределение памяти:** процесс назначения адресов для размещения переменных программы, а также именованных и неименованных констант.
5. **Генерация и оптимизация объектного кода:** процесс формирования программы на выходном языке, которая будет семантически эквивалентна исходной программе.

Пример работы процесса компиляции: Рассмотрим анализ строки с условным оператором: `if Sum>5 then pr:= true;`

На этапе **лексического анализа** текст разбивается на лексемы (токены) и классифицируется: `if` и `then` выделяются как **служебные слова**; `Sum` и `pr` — как **идентификаторы** (получают адрес в таблице идентификаторов); `5` и `true` — как **литералы** (получают адрес в таблице литералов); `>`, `:=` и `;` — как **служебные символы**.

На этапе **синтаксического анализа** компилятор группирует токены в более сложные конструкции: токены `Sum`, `>`, `5` распознаются как **логическое выражение**; токены `pr`, `:=`, `true`, `;` — как **оператор** (присваивания); а вся совокупность токенов объединяется в единую конструкцию — **условный оператор**.

На этапе **генерации кодов** распознанные синтаксические конструкции заменяются на соответствующие им заранее заготовленные фрагменты выходного кода (например, арифметическая операция может заменяться на шаблон вида `mov AX, <Op1>, add AX, <Op2>, mov <Result>, AX`), которые затем программно настраиваются и оптимизируются.

29. Синтаксис и семантика языка программирования. Средства описания синтаксиса. Примеры.

Синтаксис — это совокупность правил, определяющих допустимые конструкции языка, т. е. его форму.

Семантика — это совокупность правил, определяющих логическое соответствие между элементами и значением синтаксически корректных предложений, т. е. содержание языка.

30. Синтаксические диаграммы и особенности их построения. Примеры.

Ожидает заполнения.

31. Формальный язык и формальная грамматика языка. Пример.

Ожидает заполнения.

32. Классификация грамматик Хомского. Примеры грамматик 2-го и 3-го типов.

Ожидает заполнения.

33. Грамматический разбор. Дерево грамматического разбора. Пример.

Ожидает заполнения.

34. Левосторонний нисходящий грамматический разбор для грамматик 3-го типа. Пример.

Ожидает заполнения.

35. Левосторонний восходящий грамматический разбор для грамматик 3-го типа. Пример.

Ожидает заполнения.

36. Конечный автомат. Пример описания и алгоритм программной реализации.

Ожидает заполнения.

37. Построение лексических анализаторов на конечных автоматах. Пример.

Ожидает заполнения.

38. Построение синтаксических анализаторов на конечных автоматах. Пример.

Ожидает заполнения.

39. Автомат с магазинной памятью.

Ожидает заполнения.

40. Левосторонний нисходящий грамматический разбор для грамматик 2-го типа. Пример.

Ожидает заполнения.

41. Левосторонний восходящий грамматический разбор для грамматик 2-го типа. Пример.

Ожидает заполнения.

42. LL(k) грамматики. Метод рекурсивного спуска. Пример.

Ожидает заполнения.

43. LR(k) грамматики. Операторное предшествование. Стековый метод. Пример.

Ожидает заполнения.

44. LR(k) грамматики. Стековый метод. Пример.

Ожидает заполнения.

45. LR(k) грамматики. Польская запись. Алгоритм Бауэра-Замельзона. Пример.

Ожидает заполнения.

46. Способы распределения памяти под переменные. Достоинства и недостатки.

Ожидает заполнения.

47. Машинно-зависимая оптимизация кодов. Примеры.

Ожидает заполнения.

48. Машинно-независимая оптимизация кодов. Примеры.

Ожидает заполнения.